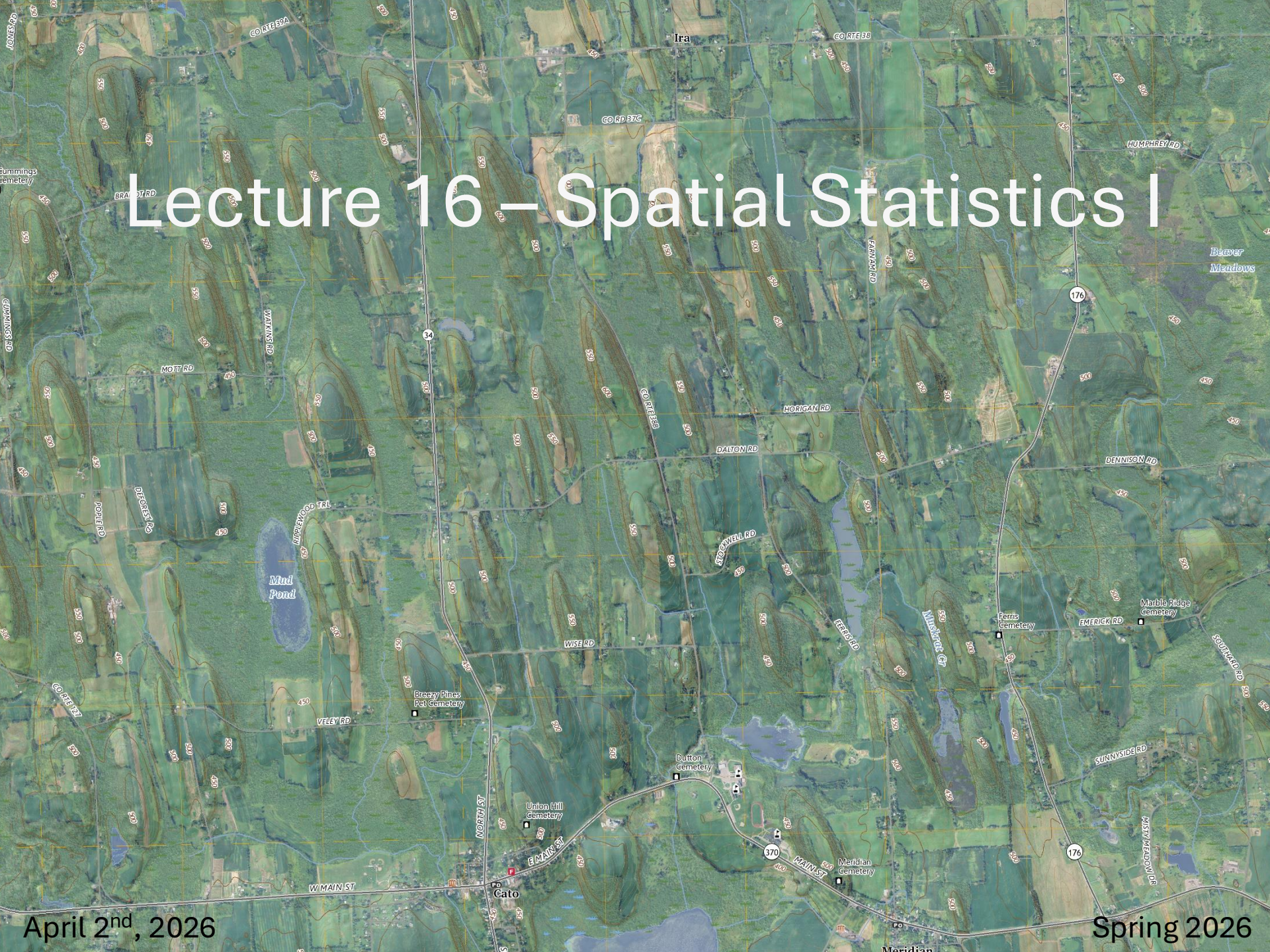
Lecture 16 – Spatial Statistics I



April 2nd, 2026

Spring 2026

Today's Learning Outcomes

- Visualize spatial data using Python mapping tools
  - *(geopandas, cartopy, xarray)*
- Understand different types of spatial data distributions
  - *(regular, clumped/clustered, random)*
- Test a spatial hypothesis using distance-based methods
  - *(correlation of distance vs. variable of interest)*
- Apply G and F functions to quantify point pattern type
  - *(nearest-neighbor statistics in Python)*

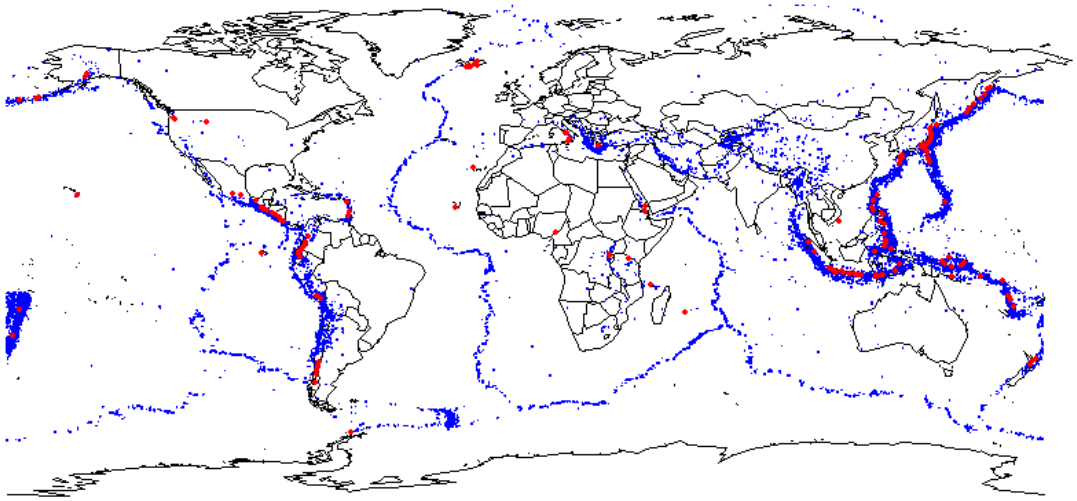
Spatial Data in Earth Sciences

Earth sciences ask not just when, but where

Three main spatial data types:

- Point data — earthquake epicenters, sample sites
- Vector data — fault lines, watershed polygons
- Raster/Grid — temperature fields, elevation DEMs

Python has excellent tools for all three types



Earthquakes (blue) and volcanoes (red) — classic spatial pattern

Python Ecosystem for Spatial Analysis

geopandas	Vector data — points, lines, polygons (shapefiles, GeoJSON)
cartopy	Map projections and basemaps with matplotlib
xarray	Gridded / raster data (NetCDF, climate, elevation)
scipy / pointpats	Spatial statistics — nearest-neighbor, G/F functions
shapely	Geometric operations — buffer, intersect, dissolve

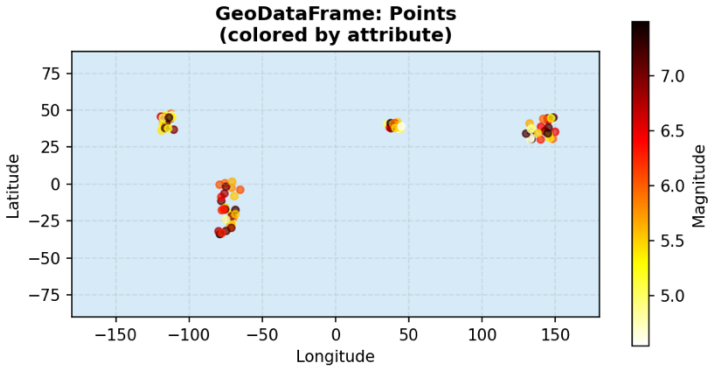
GeoPandas: Working with Vector Data

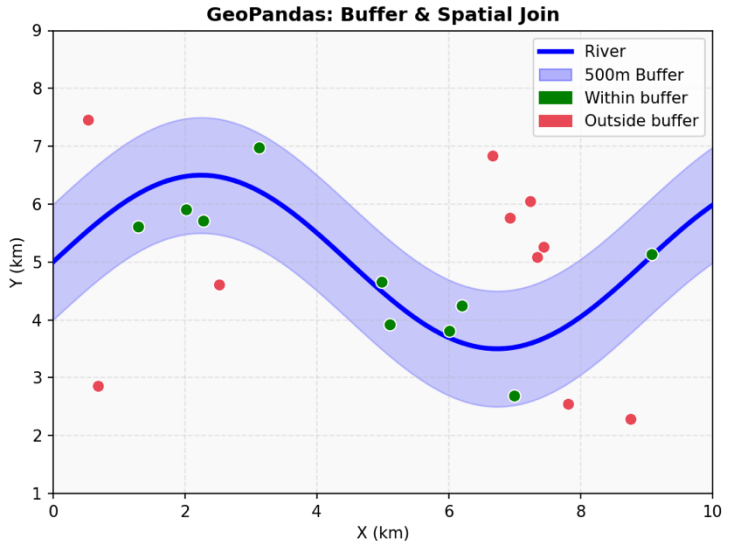
Extends pandas with geometry support

- Reads shapefiles, GeoJSON, PostGIS databases
- Supports coordinate reference systems (CRS)

Key operations:

- .plot() — quick visualization
- .buffer() — create buffer zones
- .sjoin() — spatial join
- .distance() — pairwise distances





GeoPandas: Code Example

```
import geopandas as gpd
import pandas as pd
from shapely.geometry import Point

# Load earthquake data (CSV with lat/lon columns)
df = pd.read_csv('earthquakes.csv')
gdf = gpd.GeoDataFrame(
    df, crs='EPSG:4326',
    geometry=gpd.points_from_xy(df.longitude, df.latitude)
)

# Load a world map (Natural Earth)
world = gpd.read_file(gpd.datasets.get_path('naturalearth_lowres'))

# Plot earthquakes on world map
ax = world.plot(color='lightgray', edgecolor='white', figsize=(12, 6))
gdf.plot(ax=ax, color='red', markersize=5, alpha=0.5, label='Earthquakes')
ax.set_title('Global Earthquake Locations')
```

Cartopy: Map Projections and Basemaps

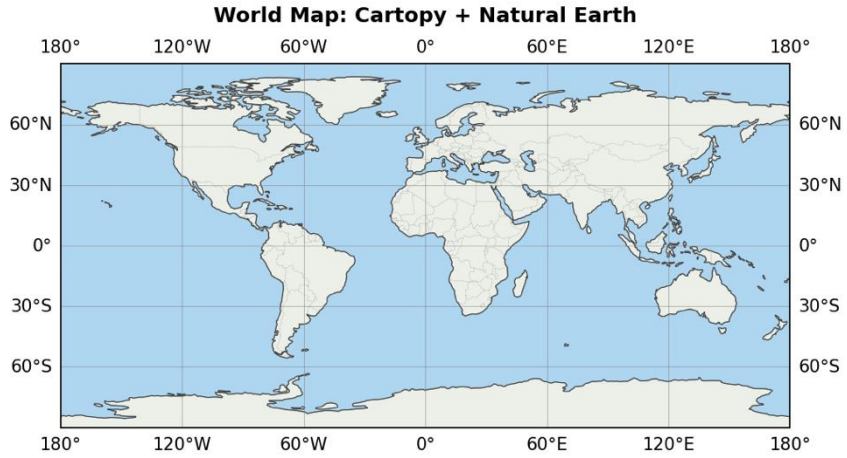
Built on matplotlib; adds geospatial projections

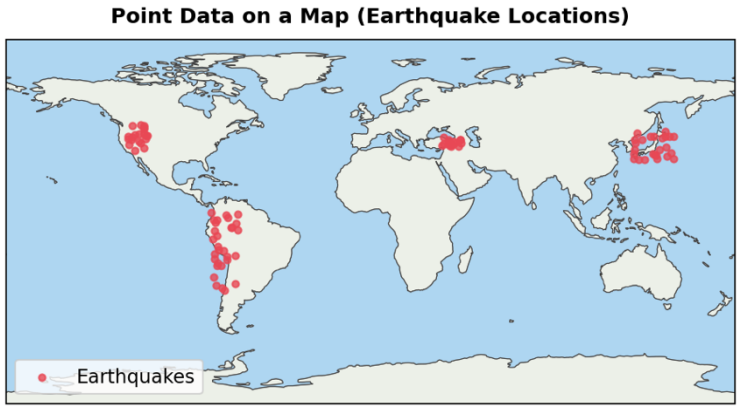
Common projections:

- PlateCarree — simple lat/lon grid
- Mollweide — equal-area global
- LambertConformal — regional maps

Natural Earth features:

- OCEAN, LAND, COASTLINE, BORDERS, RIVERS
- Easily combined with geopandas data





Cartopy: Code Example

```
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 6),
    subplot_kw={'projection': ccrs.PlateCarree()})

# Add Natural Earth background features
ax.add_feature(cfeature.OCEAN, facecolor='lightblue')
ax.add_feature(cfeature.LAND, facecolor='lightgreen')
ax.add_feature(cfeature.COASTLINE, linewidth=0.5)
ax.add_feature(cfeature.BORDERS, linestyle=':', linewidth=0.4)
ax.gridlines(draw_labels=True)

# Overlay data points (e.g., earthquake locations)
ax.scatter(eq_lons, eq_lats, c='red', s=10, alpha=0.6,
    transform=ccrs.PlateCarree(), label='Earthquakes')
ax.set_global()
ax.set_title('Earthquake Locations (Cartopy)')
plt.legend()
```

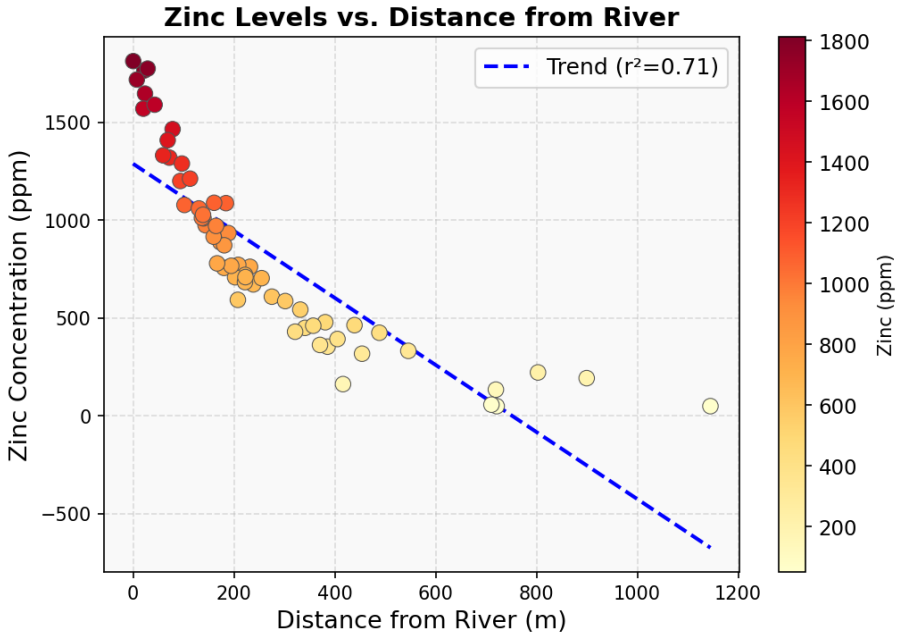
Spatial Distributions

- **Geology asks: does location predict a variable?**
  - Volcanic activity and distance from plate boundaries
  - Contaminant concentration and distance from a source
  - Ore deposit density and structural geology features
- Two types of spatial questions:
  - Variable relationship — does value change with location?
  - Directional pattern — do points cluster along a direction?

Example: Meuse River Contamination

Inspired by the classic Meuse River dataset

- Sampling locations along a river
- Zinc, lead, cadmium concentrations at each site
- Coordinates for the river path and survey area



Hypothesis

- River is the primary source of zinc contamination
- Zinc level decreases with distance from river

Statistical test:

- Correlation: distance vs. zinc concentration

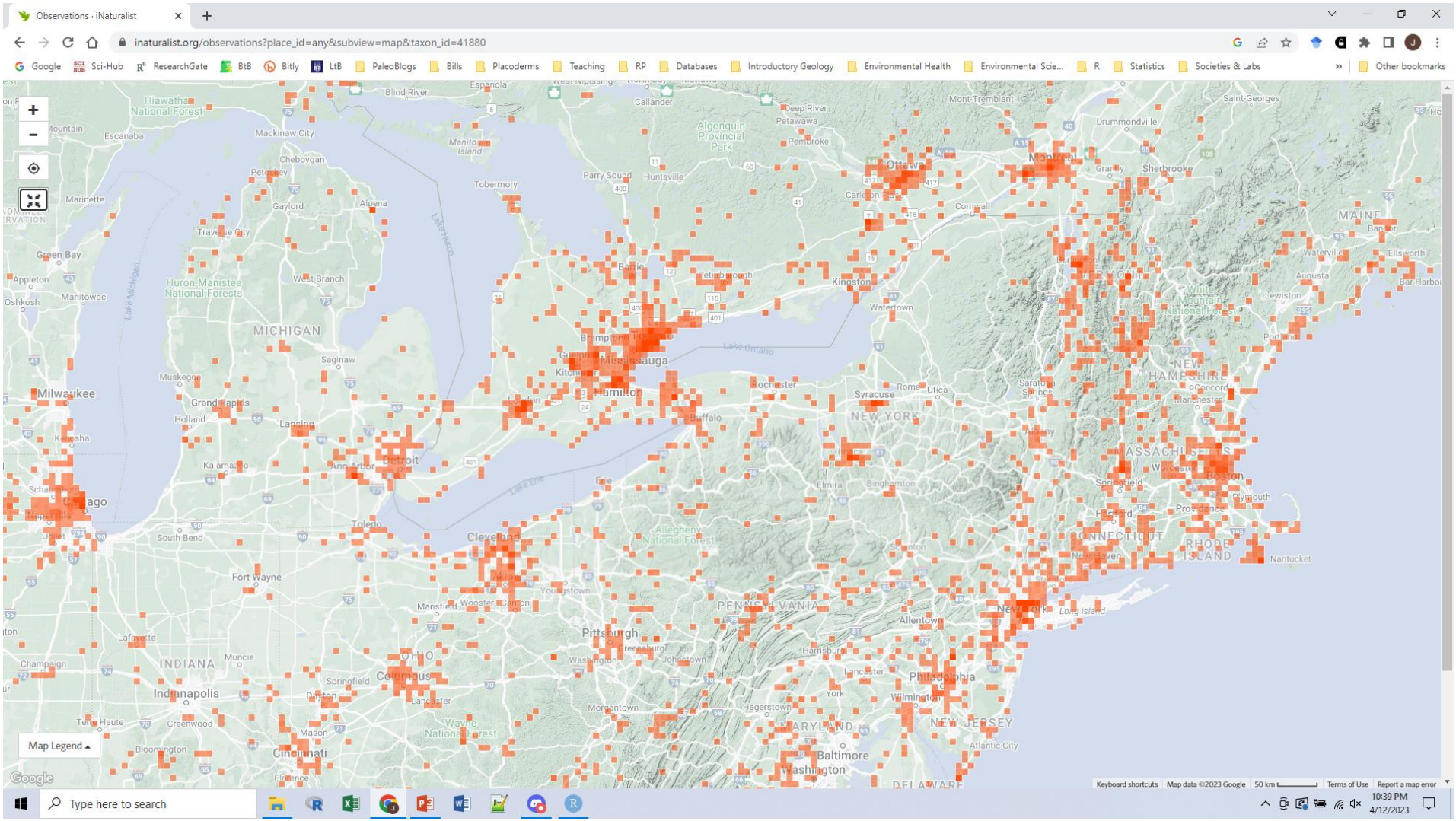
Testing a Spatial Hypothesis: Distance vs. Variable

```
import geopandas as gpd
import numpy as np
from scipy.stats import pearsonr
import matplotlib.pyplot as plt

# Sample locations as GeoDataFrame
samples = gpd.GeoDataFrame(df,
    geometry=gpd.points_from_xy(df.x, df.y), crs='EPSG:28992')

# River as a LineString
river = gpd.GeoDataFrame(geometry=[river_line], crs='EPSG:28992')

# Compute distance from each sample to the river
samples['dist_river'] = samples.geometry.apply(
    lambda pt: river.geometry.iloc[0].distance(pt))

# Test correlation: distance vs zinc
r, p = pearsonr(samples['dist_river'], np.log(samples['zinc']))
print(f'r = {r:.3f}, p = {p:.4f}')

# Scatter plot
plt.scatter(samples['dist_river'], samples['zinc'],
    c=samples['zinc'], cmap='YlOrRd', edgecolors='gray')
plt.xlabel('Distance from River (m)')
plt.ylabel('Zinc (ppm)')
```

Aggregation of Points in Space

- **Points are rarely distributed randomly in nature**
- Clumped distributions are very common — and problematic
  - Results may reflect sampling bias, not true patterns
  - Apparent clustering near cities = observation bias
- **Why it matters:**
  - Violates independence assumption in many statistics
  - Must identify pattern type before choosing a test
  - Random (null) is the baseline to test against

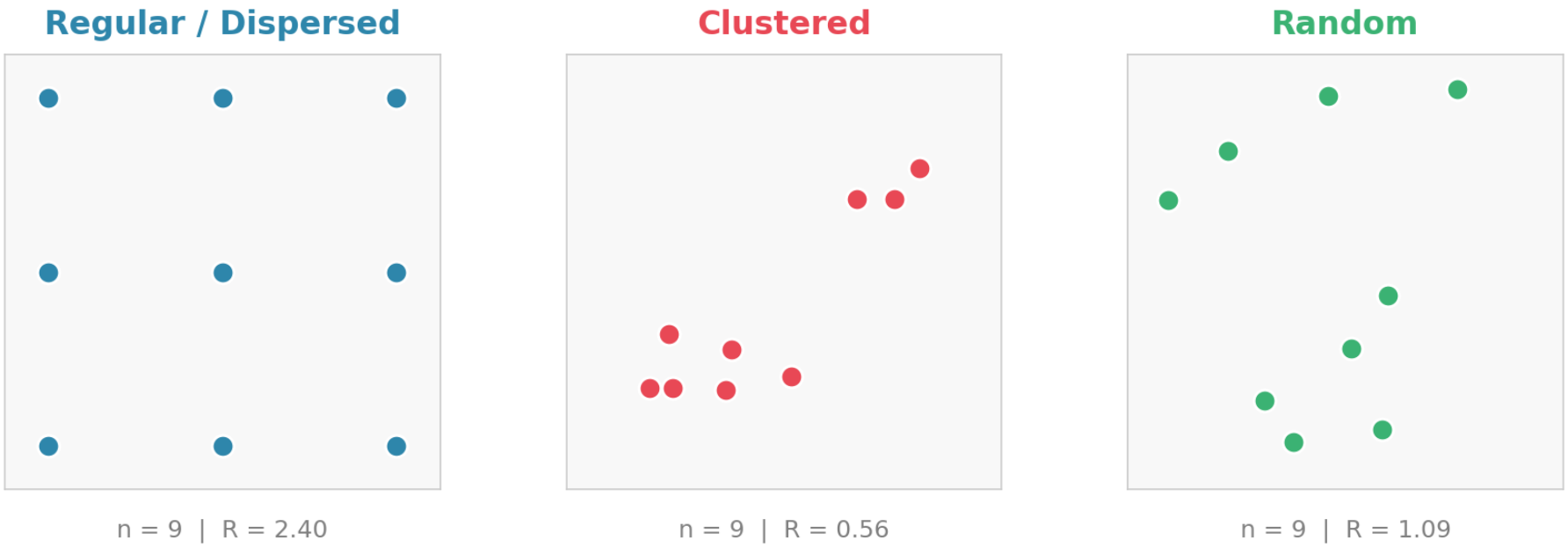
Sampling Bias in Spatial Data



iNaturalist striped skunk occurrences — many more near cities.

Reading raw data alone could lead to the false conclusion that skunks are urban specialists.

Types of Spatial Distributions



We need a quantitative way to distinguish these — and to test against random (our null hypothesis).

Testing Against Random (Null Model)

- **Complete Spatial Randomness (CSR) = null hypothesis**
- Predictions for clumped vs. dispersed:
- **Nearest-neighbor distance**
  - Clumped → shorter avg distance than CSR
  - Dispersed → longer avg distance than CSR
- **Proportion of space filled**
  - Clumped → smaller proportion than CSR
  - Dispersed → larger proportion than CSR

Aggregation Index: Clark-Evans R

- Compares observed mean nearest-neighbor distance to CSR expectation

$$R = \frac{\overline{d_{obs}}}{\overline{d_{exp}}}$$

$$\overline{d_{exp}} = \frac{1}{2\sqrt{n/A}}$$

  - $\overline{d_{obs}}$ = mean observed nearest-neighbor distance
  - n = number of points, A = study area

- **R ≈ 1 → random (CSR)**
- **R < 1 → clustered (R → 0 as clumping increases)**
- **R > 1 → dispersed (R_max ≈ 2.149: perfect hexagonal lattice)**

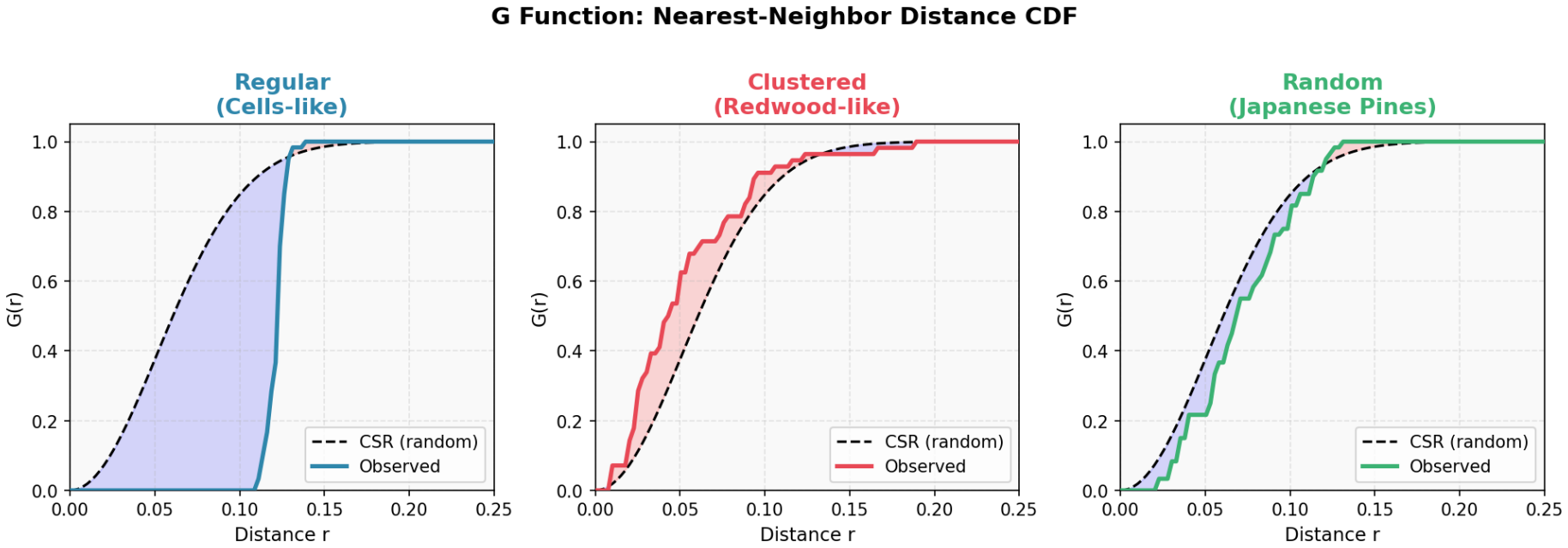
G Function: Nearest-Neighbor CDF

- **G(r) = probability that nearest-neighbor distance ≤ r**
- Cumulative distribution of distances to the closest event
- *Under CSR (random):* $G(r) = 1 - e^{-\lambda \pi r^2}$
- **Interpreting deviations from CSR:**
  - Observed G(r) > CSR → short distances are common → clustered
  - Observed G(r) < CSR → short distances are rare → dispersed

G Function: Python Code

```
import numpy as np
from scipy.spatial.distance import cdist
import matplotlib.pyplot as plt

def g_function(points, r_max=0.3, n_bins=100):
    """Empirical G(r): CDF of nearest-neighbor distances."""
    D = cdist(points, points)
    np.fill_diagonal(D, np.inf)          # exclude self-distance
    nnd = D.min(axis=1)                   # nearest-neighbor distances
    r = np.linspace(0, r_max, n_bins)
    G = np.array([np.mean(nnd <= ri) for ri in r])
    return r, G

# CSR theoretical curve
def g_csr(r, intensity):
    return 1 - np.exp(-intensity * np.pi * r**2)

r, G_obs = g_function(points)
G_rand  = g_csr(r, intensity=len(points))

plt.plot(r, G_rand, 'k--', label='CSR (random)')
plt.plot(r, G_obs,  'r-',  label='Observed')
plt.xlabel('r'); plt.ylabel('G(r)'); plt.legend()
```

G Function: Interpreting Results



Regular (cells-like): G(r) below CSR — few short-range neighbors.

Clustered (redwood): G(r) above CSR — many short-range neighbors.

Random (pines): G(r) closely tracks CSR.

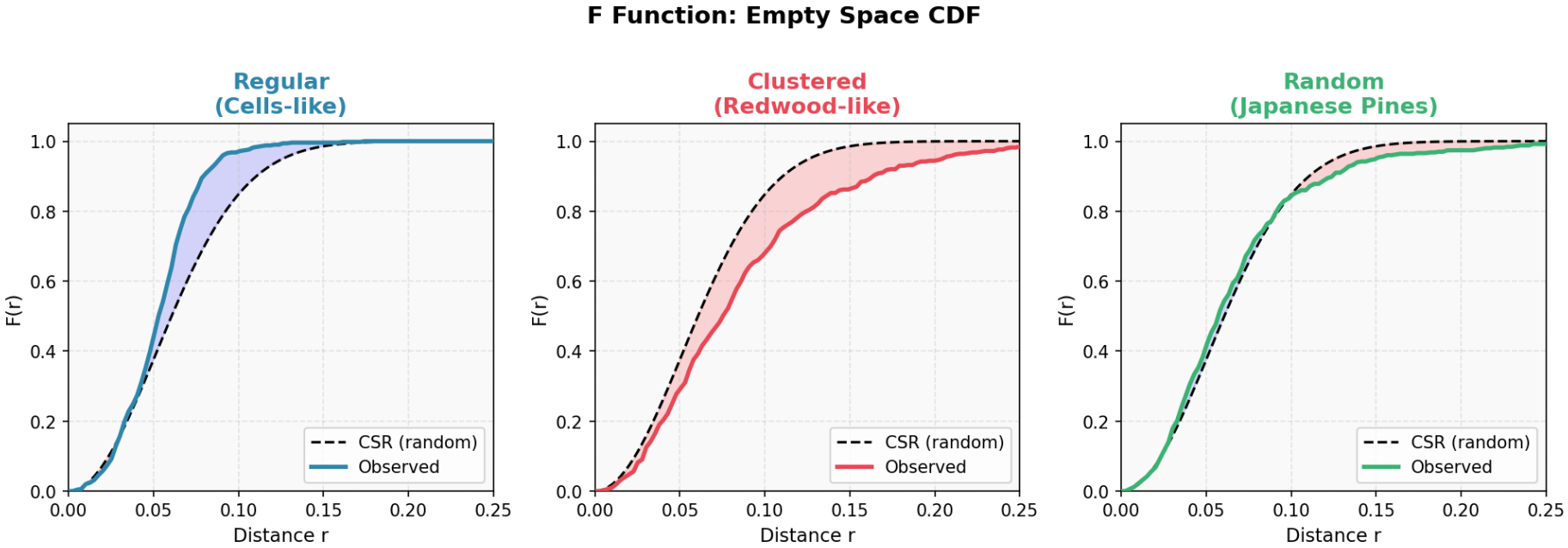
F Function: Empty Space CDF

- **F(r) = probability that distance from a random point to nearest event ≤ r**
- Also called the "empty space" function
- Complementary to G — measures unfilled space
- **Interpreting deviations from CSR:**
  - Observed F(r) < CSR → large empty gaps → clustered
  - Observed F(r) > CSR → space is well filled → dispersed

F Function: Python Code

```
from scipy.spatial.distance import cdist
import numpy as np

def f_function(points, r_max=0.3, n_bins=100, n_random=1000):
    """Empirical F(r): CDF of distances from random pts to nearest event."""
    rand_pts = np.random.uniform(0, 1, (n_random, 2))
    D = cdist(rand_pts, points).min(axis=1)  # nearest event for each rand pt
    r = np.linspace(0, r_max, n_bins)
    F = np.array([np.mean(D <= ri) for ri in r])
    return r, F

r, F_obs  = f_function(points)
F_rand = 1 - np.exp(-intensity * np.pi * r**2)  # same as G for CSR

plt.plot(r, F_rand, 'k--', label='CSR')
plt.plot(r, F_obs,  'b-',  label='Observed F(r)')
plt.xlabel('r'); plt.ylabel('F(r)'); plt.legend()
# F below CSR → clustered (large empty gaps)
# F above CSR → dispersed (space evenly filled)
```

F Function: Interpreting Results



Regular: F(r) above CSR — space is evenly filled.

Clustered: F(r) below CSR — large empty gaps between clusters.

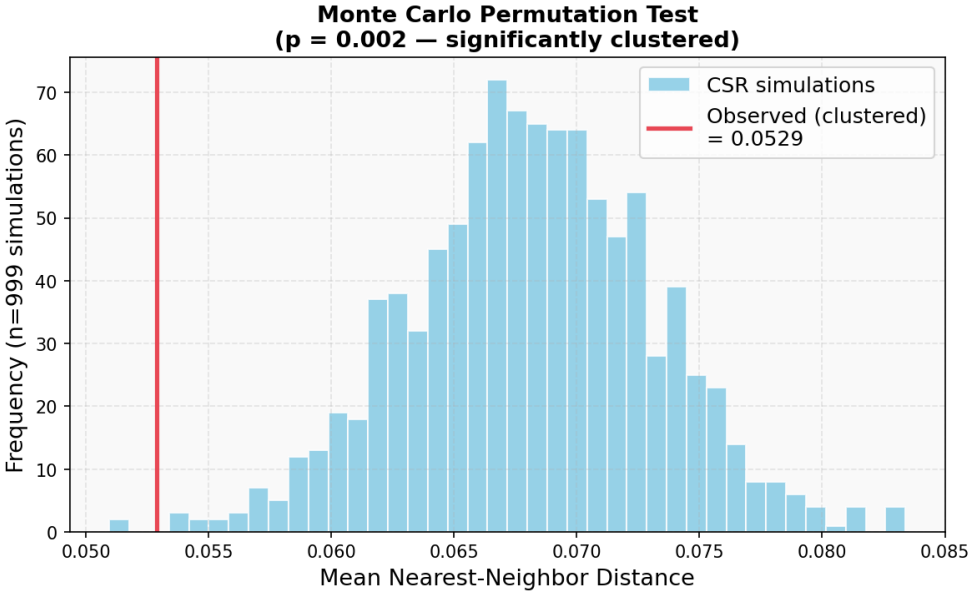
Random: F(r) closely matches CSR expectation.

Permutation (Monte Carlo) Test

Simulate the null (CSR) many times

For each simulation:

- Randomly place N points in the study area
- Compute a summary statistic (e.g., mean NND)



Compare observed statistic to simulation envelope

- p-value = fraction of simulations more extreme

Avoids parametric assumptions

- Works for any statistic, any study window shape

Complete Spatial Analysis Workflow

- **Load data with geopandas / pandas**
  - Read shapefile, CSV, or NetCDF; assign CRS
- **Visualize on a map with cartopy or geopandas**
  - Check for obvious patterns; overlay basemap
- **Formulate a spatial hypothesis**
  - Distance vs. variable, or point pattern type
- **Compute test statistic (G/F/A_i)**
  - scipy, pointpats, or manual implementation
- **Test against null with permutation**
  - Generate CSR envelopes; compute p-value

In-Class Demo: spatial_stats_demo.py

- **Part 1 — Spatial Mapping**
  - World map with cartopy + earthquake point overlay
  - GeoPandas: load data, compute buffer, spatial join
  - Xarray: plot synthetic global temperature field
- **Part 2 — Spatial Statistics**
  - River contamination: distance–zinc correlation + regression
  - G function: cells / redwood / pines point patterns
  - F function comparison against CSR
  - Monte Carlo permutation test for clustering

Today's Learning Outcomes

- **Visualize spatial data using Python mapping tools**
  - *geopandas, cartopy, xarray*
- **Understand regular, clumped, and random distributions**
  - *Aggregation Index A_i, nearest-neighbor distance*
- **Test a spatial hypothesis using distance-based methods**
  - *Correlation of distance vs. variable*
- **Apply G and F functions to characterize point patterns**
  - *Scipy implementation + Monte Carlo permutation test*